

1. Auditoría general

Evaluación del mockup actual

Área	Estado actual	Objetivo
Diseño/UI	 8.5/10	9.5/10
Flujo POS	 7/10	10/10
Inventario	 5/10	10/10
Ventas	 5/10	10/10
Compras	 2/10	10/10
Clientes	 2/10	9/10
Proveedores	 2/10	9/10
Caja	 4/10	10/10
Contabilidad	 5/10	9/10
Reportes	 4/10	10/10
Usuarios/Roles	 1/10	10/10
Seguridad	 1/10	10/10
Multiempresa/sucursales	 0/10	9/10
Arquitectura	 1/10	10/10
Persistencia	 0/10	10/10
Auditoría	 4/10	10/10

Mi valoración como producto

Mockup: 6.0/10

Potencial del sistema final: 9.5/10

Y hay una diferencia importante:

El mockup no está mal. El problema es que actualmente representa más una demostración del sistema que un sistema empresarial real.

Eso es exactamente lo que debemos cambiar.

2. Lo que ya tienes bien planteado

Hay bastante trabajo útil en el diseño.

Tienes una estructura general de módulos bastante correcta:

- Dashboard
- Punto de Venta
- Ventas
- Productos
- Inventario
- Compras
- Proveedores
- Clientes
- Caja
- Contabilidad
- Reportes
- Usuarios
- Configuración

El menú ya refleja prácticamente el esqueleto de un ERP/POS pequeño o mediano.

También tienes una separación conceptual bastante buena entre:

venta → inventario → caja → contabilidad → reportes.

Eso es importante porque esa será precisamente la columna vertebral del sistema definitivo.

3. El POS está bien encaminado

El terminal tiene:

- búsqueda de productos
- búsqueda por código

- categorías
- productos
- carrito
- cantidades
- descuento
- ITBIS
- total
- checkout
- método de pago
- efectivo recibido
- cambio
- cobro

Por ejemplo, actualmente el POS permite buscar por nombre o código de producto y agregar artículos al carrito.

También tienes el cálculo de:

Subtotal → descuento → ITBIS → total.

Eso lo conservaría conceptualmente.

Pero el POS real debe crecer muchísimo.

Yo agregaría:

Venta

- venta rápida
- venta por código de barras
- búsqueda por nombre
- búsqueda por SKU
- búsqueda por categoría
- búsqueda por marca
- cliente consumidor final
- selección de cliente

- venta a crédito
- múltiples métodos de pago
- pago mixto
- descuentos por línea
- descuento global
- promociones
- impuestos configurables
- productos exentos
- productos gravados
- productos con diferentes tasas
- precio especial
- precio mayorista
- devolución
- cambio de producto
- anulación
- suspensión de venta
- recuperar venta suspendida
- cotización
- convertir cotización → venta
- impresión de factura
- reimpresión
- correo/WhatsApp de factura
- notas de crédito
- historial del cliente

4. Hay un problema importante con la lógica actual

Actualmente completeSale() modifica directamente el estado del navegador:

state.cart

state.products

state.auditLogs

state.accountingEntries

Por ejemplo, al completar una venta reduce el stock directamente en memoria y genera un número de factura aleatorio.

Esto **no puede existir en producción**.

El flujo correcto será:

React

↓

POST /api/v1/sales

↓

FastAPI

↓

Sales Service

↓

SQL Server Transaction

|— Sale

|— SaleDetails

|— Payment

|— InventoryMovement

|— CashMovement

|— AccountingEntry

|— AuditLog

Todo debe ocurrir dentro de una **transacción SQL**.

Si algo falla:

ROLLBACK

Si todo funciona:

COMMIT

Esto evita situaciones como:

factura creada pero inventario no actualizado.

o:

inventario descontado pero venta no registrada.

5. El inventario debe ser mucho más avanzado

Ahora mismo tienes básicamente:

Producto

Categoría

Precio

Stock

Código

El mockup incluso tiene un ajuste directo mediante `prompt()`.

Eso debe desaparecer.

Inventario profesional

Yo diseñaría:

Productos

- SKU
- código de barras
- nombre
- descripción
- categoría
- subcategoría
- marca
- unidad de medida
- costo
- precio
- precio mayorista

- precio mínimo
- margen
- impuesto
- proveedor principal
- stock mínimo
- stock máximo
- punto de reorden
- ubicación
- imagen
- producto activo/inactivo

Inventario

- existencia actual
- existencia disponible
- existencia reservada
- entradas
- salidas
- ajustes
- transferencias
- devoluciones
- pérdidas
- vencimientos
- lotes
- números de serie

Kardex

Cada movimiento debería guardar:

Fecha

Producto

Tipo movimiento

Cantidad

Costo

Existencia anterior

Existencia nueva

Usuario

Sucursal

Documento origen

Motivo

6. Agregaría multi-sucursal desde el principio

Aunque inicialmente tengas una sola tienda, **diseñaría MSSQL para soportar múltiples sucursales.**

Por ejemplo:

Empresa

|

|— Sucursal 1

|

|— Caja 1

|

|— Caja 2

|

└─ Inventario

|

|— Sucursal 2

|

|— Caja 3

|

└─ Inventario

|

└─ Sucursal 3

Esto cambia completamente el potencial comercial.

El mismo software podría venderse a:

- colmados

- minimarkets
 - supermercados pequeños
 - farmacias
 - tiendas
 - ferreterías
 - restaurantes
 - distribuidores
 - negocios con varias sucursales.
-

7. Caja necesita una reconstrucción completa

Ahora mismo Caja básicamente cambia entre:

Abierta

Cerrada

mediante una variable JavaScript.

Eso es solamente un mock.

La caja real debería manejar:

Apertura

Caja

Usuario

Sucursal

Fecha

Fondo inicial

Durante la jornada

- ventas efectivo
- ventas tarjeta
- transferencias
- devoluciones
- gastos

- entradas manuales
- retiros
- depósitos
- movimientos

Cierre

Sistema calcula:

Fondo inicial

+ ventas efectivo

+ entradas

- devoluciones

- retiros

- gastos

= efectivo esperado

Luego:

Efectivo contado

-

Efectivo esperado

=

Diferencia

Y registra:

sobrante / faltante.

8. Métodos de pago

El mockup solamente plantea:

- efectivo
- tarjeta
- transferencia

Yo lo convertiría en:

EFFECTIVO

TARJETA

TRANSFERENCIA

CHEQUE

CRÉDITO

OTRO

Pero lo más importante:

Pago mixto

Ejemplo:

Venta:

RD\$ 5,000

Cliente paga:

Efectivo RD\$ 2,000

Tarjeta RD\$ 3,000

Esto es extremadamente útil en POS reales.

9. Clientes

Actualmente es prácticamente un placeholder.

Yo agregaría:

Clientes

|

└─ Datos generales

└─ Documento/RNC/Cédula

└─ Teléfono

└─ Email

└─ Dirección

└─ Límite de crédito

└─ Balance

└— Historial de compras

└— Facturas

└— Pagos

└— Devoluciones

└— Estado de cuenta

Y algo comercialmente interesante:

Programa de fidelización

- puntos
- niveles
- descuentos
- clientes frecuentes
- promociones personalizadas.

10. Proveedores

Debe relacionarse directamente con:

Proveedor

↓

Orden de compra

↓

Recepción

↓

Inventario

↓

Cuenta por pagar

↓

Pago

No solamente registrar proveedores.

11. Compras

Este es uno de los módulos que más necesita desarrollo.

Diseñaría:

Orden de compra

OC-000001

Proveedor

Fecha

Sucursal

Usuario

Productos

Cantidades

Costos

Impuestos

Total

Estado

Estados:

BORRADOR

PENDIENTE

APROBADA

RECIBIDA

PARCIAL

CANCELADA

Y posteriormente:

Recepción de mercancía.

Esto permitirá que comprar mercancía incremente correctamente el inventario.

12. Contabilidad: aquí tienes una buena idea, pero hay que profesionalizarla

Tu mockup ya intenta generar asientos automáticos.

Por ejemplo, para una venta tienes:

Caja

Débito

Ingresos por Ventas

Crédito

ITBIS por Pagar

Crédito

y también contemplas costo de ventas e inventario en otros movimientos.

Eso es correcto conceptualmente.

Pero yo no dejaría la contabilidad como un módulo aislado.

Debe funcionar como:

VENTA

↓

CONTABILIDAD

COMPRA

↓

CONTABILIDAD

PAGO

↓

CONTABILIDAD

GASTO

↓

CONTABILIDAD

DEVOLUCIÓN



CONTABILIDAD

Es decir:

la contabilidad debe alimentarse automáticamente de las operaciones del sistema.

13. Mejoraría el módulo contable

Actualmente tienes:

- Libro Diario
- Catálogo
- Balance de comprobación
- Estado de resultados
- Balance general

Eso está muy bien como base.

Agregaría:

Contabilidad

- catálogo de cuentas configurable
- períodos contables
- cierre de períodos
- asientos automáticos
- asientos manuales
- cuentas por cobrar
- cuentas por pagar
- conciliación
- libro mayor
- balance de comprobación

- estado de resultados
 - balance general
 - flujo de efectivo
 - depreciaciones
 - gastos
 - centros de costo
 - historial de modificaciones
-

14. Reportes

Tu concepto actual contempla PDF, Excel y CSV.

Pero actualmente son básicamente simulaciones; por ejemplo, la función de exportación genera un CSV fijo y para PDF/Excel solamente muestra un mensaje.

En producción lo llevaría a:

Ventas

- ventas por día
- ventas por hora
- ventas por usuario
- ventas por sucursal
- ventas por producto
- ventas por categoría
- ventas por cliente
- ventas por método de pago
- anulaciones
- devoluciones
- descuentos

Inventario

- inventario actual
- kardex

- valorización
- bajo stock
- sobrestock
- productos sin movimiento
- productos próximos a vencer
- productos vencidos
- pérdidas

Finanzas

- cuentas por cobrar
 - cuentas por pagar
 - flujo de caja
 - utilidad
 - margen
 - gastos
 - impuestos
-

15. Dashboard

El Dashboard visualmente está bastante bien.

Ya tienes:

- ventas del día
- semana
- mes
- año
- utilidad
- ticket promedio
- valor de inventario
- caja
- ventas últimos 7 días

- productos más vendidos
- ventas por hora
- alertas de stock
- vencimientos
- productos sin movimiento
- actividad reciente.

Eso es excelente como concepto.

Pero todos esos valores actualmente son datos de demostración.

Por ejemplo, las gráficas contienen valores escritos directamente en JavaScript.

La versión final debería obtenerlos mediante:

GET /api/v1/dashboard

y FastAPI consulta SQL Server.

16. Seguridad: uno de los cambios más importantes

Actualmente el usuario está literalmente definido dentro del HTML:

Elias Hughes

Administrador

y las operaciones utilizan incluso el usuario directamente en JavaScript.

Eso debe desaparecer completamente.

Implementaría:

Autenticación

Login

↓

FastAPI

↓

JWT / Refresh Token

↓

React

Roles

Por ejemplo:

Super Administrador

Todo.

Administrador

Configuración + operaciones administrativas.

Gerente

Reportes + supervisión.

Contador

Contabilidad + reportes financieros.

Supervisor

Inventario + ventas + caja.

Cajero

POS + caja.

Almacén

Inventario + compras.

17. RBAC granular

No solamente:

admin

user

Sino permisos:

sales.create

sales.cancel

sales.refund

inventory.create

inventory.adjust

inventory.transfer

cash.open

cash.close

cash.withdraw

products.create

products.edit

products.delete

accounting.create

accounting.edit

reports.view

reports.export

users.create

users.edit

users.disable

Esto permite que el administrador controle exactamente qué puede hacer cada usuario.

18. Auditoría

Tu mockup ya tiene un concepto interesante de actividad reciente.

Lo convertiría en un verdadero:

Audit Log

Cada acción importante:

Usuario

Fecha

Hora

IP

Sucursal

Módulo

Acción

Registro afectado

Valor anterior

Valor nuevo

Resultado

Ejemplo:

Elias

14/09/2026 10:45

Inventario

Producto #152

Stock

100 → 80

Motivo: Ajuste

Esto es muy importante para una solución empresarial.

19. Arquitectura que te recomiendo

Aquí es donde cambiaría radicalmente el proyecto.

Frontend

React

TypeScript

HTML

CSS

Estructura:

frontend/

|

└─ src/

|

└─ app/

|

└─ components/

|

└─ layouts/

|

└─ pages/

|

|

|

└─ modules/

|

|

└─ dashboard/

|

|

└─ pos/

|

|

└─ sales/

|

|

└─ products/

|

|

└─ inventory/

|

|

└─ purchases/

|

|

└─ suppliers/

|

|

└─ customers/

|

|

└─ cash/

|

|

└─ accounting/

|

|

└─ reports/

|

|

└─ users/

|

|

|

|

└─ services/

|

|

└─ hooks/

|

|

└─ types/

|

|

└─ utils/

|

|

└─ routes/

20. Backend FastAPI

No haría:

main.py

5,000 líneas

Lo dividiría:

backend/

|

|— app/

| |— main.py

| |

| |— core/

| | |— config.py

| | |— security.py

| | |— database.py

| | |— logging.py

| |

| |— models/

| | |— user.py

| | |— product.py

| | |— sale.py

| | |— inventory.py

| | |— purchase.py

| | |— customer.py

| | |— supplier.py

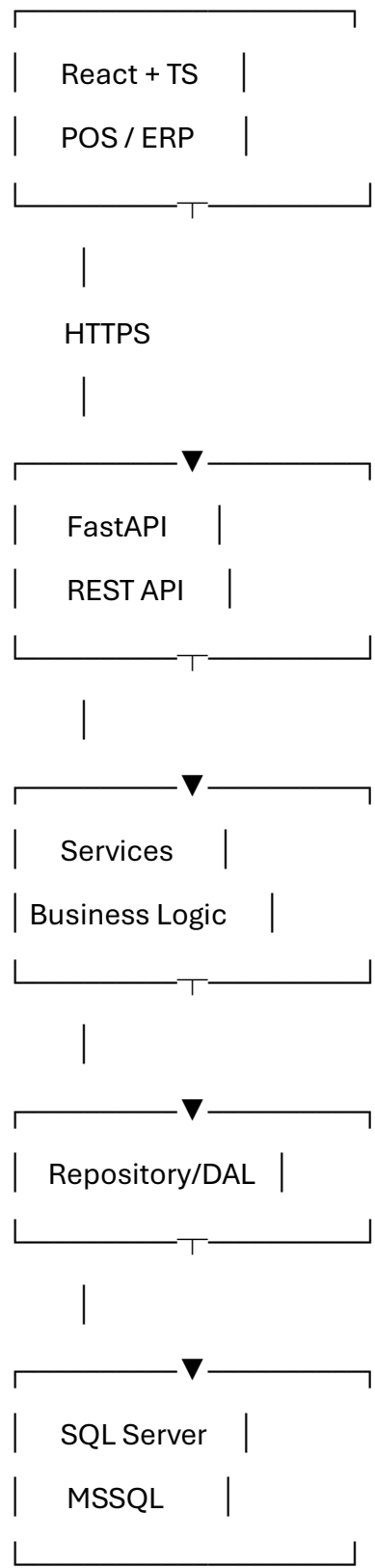
| | |— cash.py

| | |— accounting.py

```
| |
| |─ schemas/
| |
| |─ repositories/
| |
| |─ services/
| |
| |─ api/
| |  └─ v1/
| |     └─ auth.py
| |     └─ products.py
| |     └─ sales.py
| |     └─ inventory.py
| |     └─ purchases.py
| |     └─ customers.py
| |     └─ suppliers.py
| |     └─ cash.py
| |     └─ accounting.py
| |     └─ reports.py
| |     └─ dashboard.py
| |
| |  └─ utils/
|
|─ tests/
|─ migrations/
|─ requirements.txt
└─ .env
```

21. Arquitectura lógica

La aplicación debería quedar:



22. Base de datos MSSQL

Yo no intentaría crear una sola tabla gigante.

Un esquema inicial podría ser:

Users

Roles

Permissions

UserRoles

RolePermissions

Companies

Branches

Terminals

Products

ProductCategories

ProductBrands

ProductTaxes

ProductPrices

Customers

CustomerAddresses

Suppliers

SupplierAddresses

Sales

SaleDetails

SalePayments

SaleTaxes

Purchases

PurchaseDetails

PurchasePayments

Inventory

InventoryMovements

InventoryTransfers

InventoryAdjustments

CashRegisters

CashSessions

CashMovements

Accounts

JournalEntries

JournalEntryDetails

AccountsReceivable

AccountsPayable

Quotes

QuoteDetails

Returns

ReturnDetails

AuditLogs

SystemSettings

23. Algo que considero MUY importante: documentos y estados

No debemos simplemente hacer CRUD.

Cada proceso debe tener un ciclo de vida.

Ejemplo:

Venta

BORRADOR

↓

CONFIRMADA

↓

PAGADA

↓

FACTURADA

o:

ANULADA

DEVUELTA

Compra

BORRADOR

↓

APROBADA

↓

RECIBIDA

↓

CONTABILIZADA

↓

PAGADA

Esto hace que el sistema sea realmente empresarial.

24. Facturación dominicana

Como estás pensando comercializarlo en República Dominicana, yo dejaría la arquitectura preparada desde el principio para **facturación fiscal/NFC/e-CF y DGII**, pero no mezclaría la lógica fiscal directamente dentro del POS.

Crearíamos una capa:

Billing Engine

que permita:

Consumidor final

Crédito fiscal

Gubernamental

Especial

Exportación

y posteriormente integrar el mecanismo fiscal correspondiente.

Esto evita tener que reconstruir el POS cuando quieras convertirlo en una solución comercial formal.

25. Funcionalidades que yo agregaría y que pueden diferenciar tu producto

Aquí está lo interesante.

Inteligencia comercial

Productos

Producto más vendido

Producto menos vendido

Producto sin movimiento

Producto con mejor margen

Producto con peor margen

Predicción de inventario

Por ejemplo:

"El arroz probablemente se agotará en 5 días según el promedio de ventas."

Reorden automático

Stock < punto de reorden



Sugerir compra

26. Alertas

Centro de notificaciones:

-  Producto agotado
 -  Stock bajo
 -  Producto próximo a vencer
 -  Cuenta por cobrar vencida
 -  Cuenta por pagar próxima
 -  Caja con diferencia
 -  Compra recibida
-

27. Gestión de promociones

Agregaría:

2x1

3x2

Descuento %

Descuento RD\$

Precio especial

Combo

Oferta por horario

Oferta por cliente

Oferta por categoría

Esto puede ser muy atractivo para colmados/minimarkets.

28. Sistema de respaldo

Para una solución local:

SQL Server



Backup automático



Backup diario



Backup semanal

Y desde configuración:

Último backup

Próximo backup

Estado

Restaurar

Exportar backup

29. Multiempresa

Si quieres convertirlo posteriormente en SaaS, agregaría desde ahora:

Tenant / Company

Todas las tablas importantes pueden tener:

company_id

branch_id

Entonces podrás tener:

Empresa A

└─ Santo Domingo

└─ Santiago

Empresa B

└─ La Vega

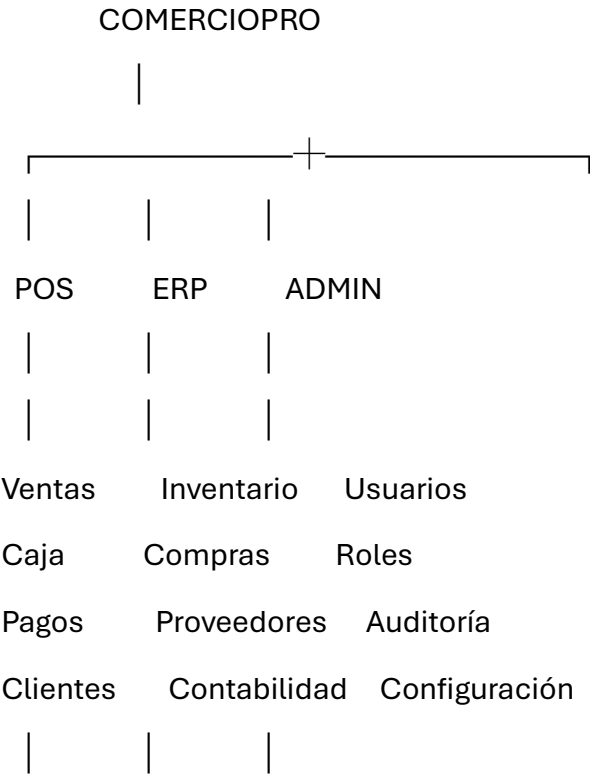
└─ Puerto Plata

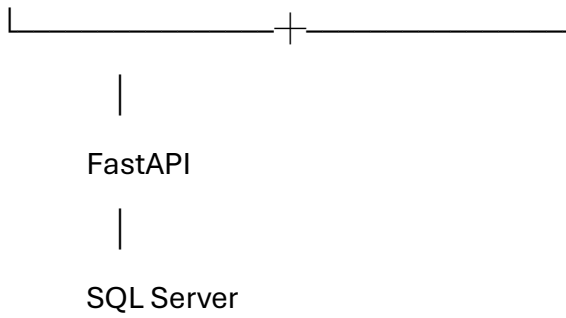
sin reconstruir todo el backend.

30. Arquitectura final que recomiendo

Yo plantearía el proyecto como:

ComercioPro





31. Plan de desarrollo que utilizaría

No intentaría construir todo al mismo tiempo.

Fase 1 — Arquitectura

Objetivo: eliminar el monolito

- React
- TypeScript
- FastAPI
- SQL Server
- estructura de carpetas
- configuración
- ORM/DAL
- migraciones
- logging
- manejo de errores

Fase 2 — Seguridad

- login
- JWT
- refresh tokens
- usuarios
- roles
- permisos

- sesiones
 - auditoría
-

Fase 3 — Maestro de datos

- empresas
 - sucursales
 - categorías
 - marcas
 - productos
 - impuestos
 - clientes
 - proveedores
-

Fase 4 — Inventario

- existencias
 - kardex
 - movimientos
 - ajustes
 - transferencias
 - mínimos/máximos
 - alertas
-

Fase 5 — POS

Aquí reconstruimos el mockup:

Buscar

↓

Producto

↓

Carrito

↓

Cliente

↓

Descuento

↓

Impuestos

↓

Pago

↓

Venta

↓

Inventario

↓

Caja

↓

Contabilidad

↓

Factura

Fase 6 — Compras

- órdenes
- recepción
- inventario
- cuentas por pagar
- pagos

Fase 7 — Caja

- apertura
 - movimientos
 - cierre
 - cuadro
 - diferencias
 - arqueo
-

Fase 8 — Contabilidad

- catálogo
 - diario
 - mayor
 - balance
 - resultados
 - integración automática
-

Fase 9 — Reportes

- ventas
 - inventario
 - caja
 - compras
 - clientes
 - proveedores
 - contabilidad
 - rentabilidad
-

Fase 10 — Funciones comerciales avanzadas

- promociones
- fidelización

- multi-sucursal
- multiempresa
- estadísticas
- alertas
- recomendaciones
- backups
- integración fiscal.

32. Lo más importante: NO reconstruiría el sistema como está

Esta sería mi decisión técnica:

 **No**

HTML



JavaScript



Todo el sistema

 **Tampoco**

React



un archivo gigante

 **Sí**

React + TypeScript



Componentes



Módulos



Servicios API

↓

FastAPI

↓

Routers

↓

Services

↓

Repositories

↓

SQL Server

33. Mi conclusión de la auditoría

El mockup **sí vale la pena conservarlo.**

De hecho, yo lo trataría como:

Documento visual y funcional de requisitos de ComercioPro 1.0.

Porque ya define una parte importante de lo que quieres: el dashboard, POS, carrito, impuestos, caja, contabilidad, inventario, reportes, usuarios y configuración.

Pero **no debemos convertirlo directamente en código de producción.**

La reconstrucción debe aprovechar:

80–90 % de la idea funcional y visual

pero aproximadamente:

0–10 % del código actual.

El código actual es excelente como **prototipo/mockup**, pero no como núcleo del ERP.

Mi objetivo para la versión nueva sería:

ComercioPro 2.0 — POS + Inventario + Compras + Caja + Contabilidad + CRM básico + Reportes

con:

React + TypeScript + FastAPI + Python + SQL Server

y una arquitectura preparada para:

una sucursal → múltiples sucursales → múltiples empresas → SaaS.

Y algo que considero especialmente importante para tu caso: **yo diseñaría la base de datos y los servicios desde el principio para que una venta sea una operación transaccional completa**, de modo que el POS no sea simplemente una pantalla que "registra ventas", sino el punto que integra automáticamente **inventario + caja + pagos + impuestos + contabilidad + auditoría.**

- ? Auditoría completa del prototipo comerciopro_pos_system.html.
- ? Resultados y calificaciones por módulo.
- ? Arquitectura propuesta **React + TypeScript + FastAPI/Python + SQL Server**.
- ? Estructura recomendada de frontend y backend.
- ? Modelo de base de datos SQL Server.
- ? POS, inventario, compras, proveedores y clientes.
- ? Caja y arqueos.
- ? Contabilidad integrada.
- ? Reportes y analítica.
- ? Usuarios, roles, permisos y auditoría.
- ? Multiempresa y multisucursal.
- ? Backups y recuperación.
- ? Transacciones SQL y control de concurrencia.
- ? Roadmap de desarrollo.
- ? Criterios para considerar el sistema listo para producción.
- ? **Módulo completo de Facturación Electrónica DGII (e-CF)**.
- ? Tipos de e-CF 31, 32, 33, 34, 41, 43, 44 y 45.
- ? Arquitectura Electronic Invoicing Service.
- ? Generación XML, firma digital, envío, respuestas, estados, secuenciales y contingencia.
- ? Tablas fiscales propuestas.
- ? Proceso de certificación DGII.
- ? Consideraciones de la **Ley 32-23** y calendario de implementación actualizado.
- ? Fuentes oficiales de la DGII.